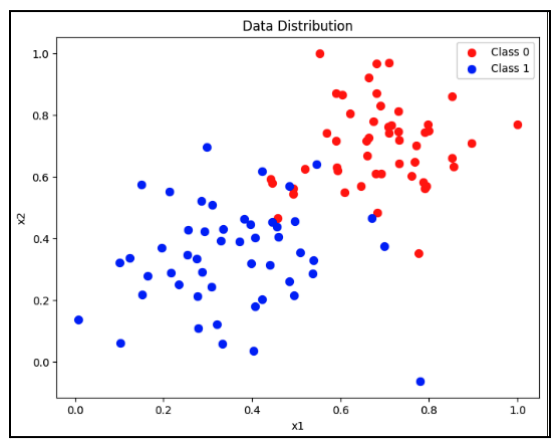
Harrison Lloyd
CSCI 485
Prof. Shen
4/17/26

Homework 5

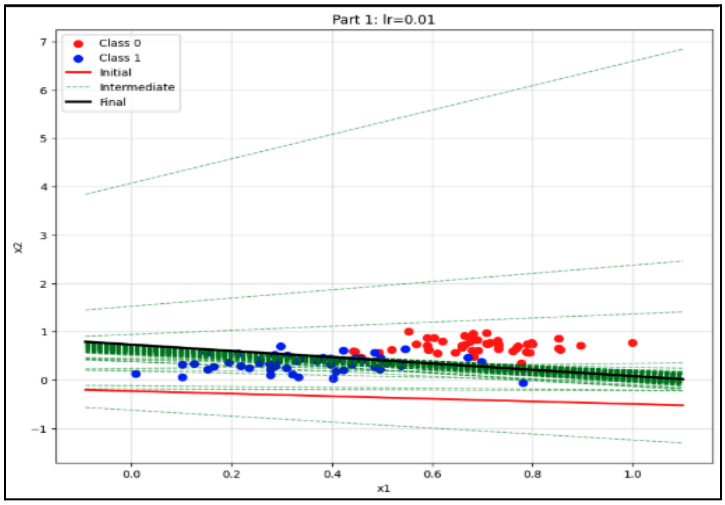
This assignment implements two different perceptron algorithms for binary classification. A perceptron is a simple neural network that learns to draw a line separating two classes of data points. Part 1 uses a heuristic approach with a step function that only updates weights when it makes a mistake. Part 2 uses gradient descent with a sigmoid activation function that gives continuous predictions and updates weights based on how wrong the prediction is. The goal is to experiment with different learning rates and see how they affect the model's performance. The dataset has 100 points with 2 features, split evenly between two classes.

```
data=pd.read_csv('data.csv',header=None,names=['x1','x2','label'])
data.head()



X=data[['x1','x2']].values
y=data['label'].values
```



The dataset has 100 samples with 50 in each class. Class 0, in the red, are mostly in the lower left area and Class 1, in the blue, are in the upper right. The data looks linearly separable.

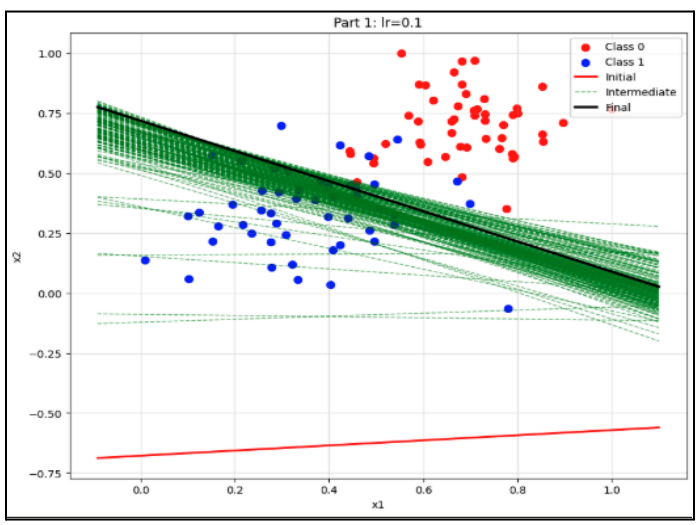
Part 1: Heuristic Approach




```
for iteration in range(self.max_iter):
    errors=0
    for i in range(n_samples):
        xi=X[i]
        yi=y[i]
        z=np.dot(xi,self.weights)+self.bias
        pred=self.step_function(z)
        if pred!=yi:
            errors+=1
            if pred==0:
                self.bias+=self.lr
                self.weights+=self.lr*xi
            else:
                self.bias-=self.lr
                self.weights-=self.lr*xi
    self.w_history.append(self.weights.copy())
    self.b_history.append(self.bias)
    if errors==0:
        break
```

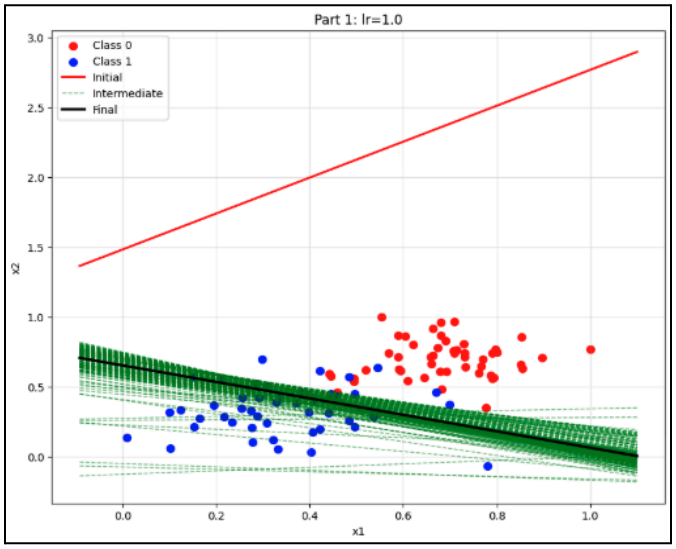
Accuracy=91%, Learning Rate=0.01

Since the lr=0.01, the learning was really slow. You can see tons of green dashed lines which means it took a lot of iterations to adjust the decision boundary. The algorithm didn't fully converge even after 200 iterations. The learning rate is too small and the weights change very slowly so it takes forever to learn.



Accuracy=93%, Learning Rate=0.1

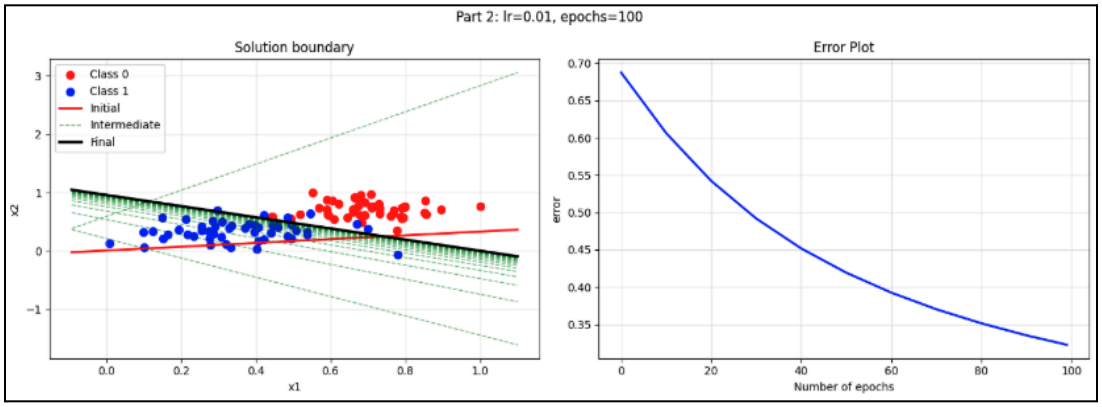
This learning rate worked the best for Part 1. The model learned faster than 0.01 and got better accuracy. The decision boundary moved more efficiently toward the optimal position. You can see fewer intermediate lines compared to lr=0.01, meaning it converged quicker.



Accuracy=91%, Learning Rate=1.0

The large learning rate gave 91% accuracy which is worse than 0.1. The weights ended up really big (i.e. -7.5 and -10.9). This happens because the learning rate is too high and the updates are too aggressive, causing the model to overshoot the optimal solution. The boundary jumps around too much instead of smoothly converging.

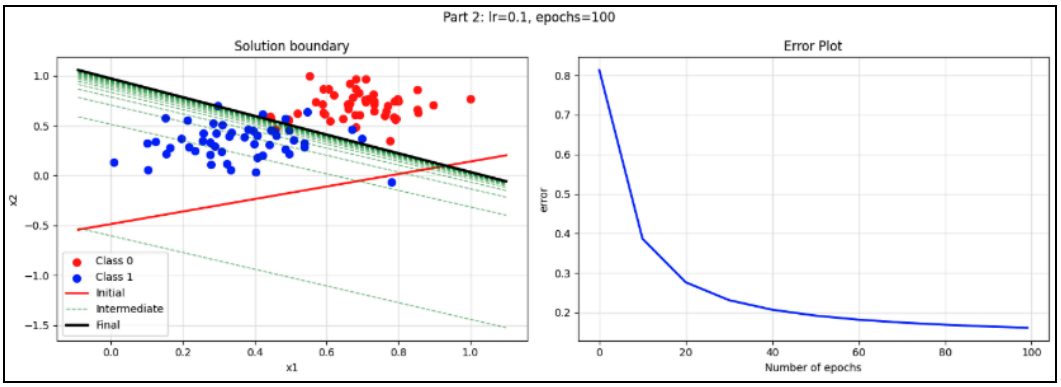
Part 2: Gradient Descent with Sigmoid



Learning Rate=0.01, Epochs=100Accuracy=93%, Final Loss=0.3223

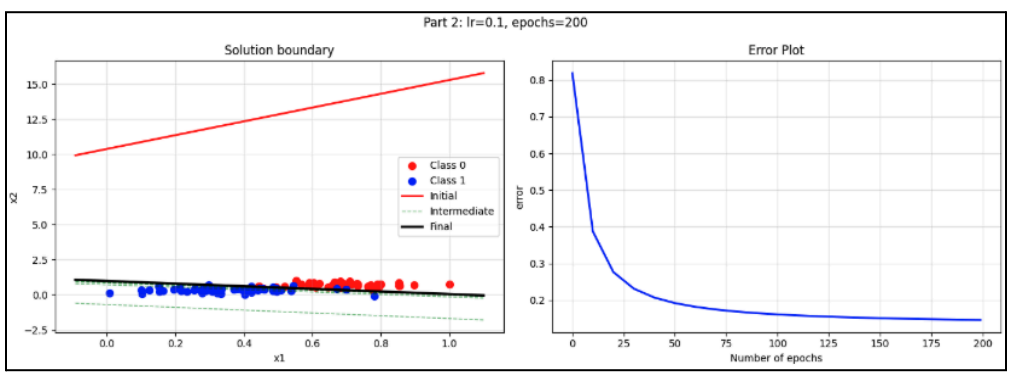
```
def sigmoid(self,z):
    z=np.clip(z,-500,500)
    return 1/(1+np.exp(-z))

def log_loss(self,y_true,y_pred):
    eps=1e-15
    y_pred=np.clip(y_pred,eps,1-eps)
    return -np.mean(y_true*np.log(y_pred)+(1-y_true)*np.log(1-y_pred))
```

The error plot shows the loss going down steadily but slowly over the 100 epochs. The loss is still pretty high at the end (0.32) which means the model could keep improving if we trained longer. The small learning rate makes learning stable but slow.



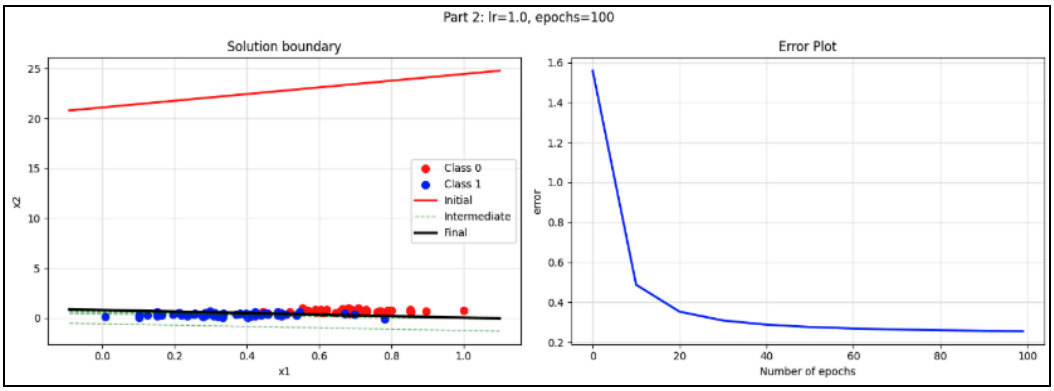
Learning Rate=0.1, Epochs=100, Accuracy=93%, Final Loss=0.1617

With lr=0.1 the loss dropped much faster. The error plot shows a steep decline at the start then it levels off. The final loss (0.16) is way better than 0.01, showing this learning rate is more efficient. The model learned the same accuracy but with much lower error.



Learning Rate=0.1, Epochs=200, Accuracy=94%, Final Loss=0.1464

This was the best configuration. By training for 200 epochs instead of 100, the model achieved 94% accuracy and the lowest loss (0.14). The error plot shows the loss keeps gradually decreasing even after 100 epochs, proving that more training helps when you have a good learning rate.



Learning Rate=1.0, Epochs=100, Accuracy=93%, Final Loss=0.2556

The error plot starts really high (loss of 1.52 at the beginning) which shows the learning rate is too big and causes instability at first. Even though it eventually stabilizes, the final loss (0.26) is worse than lr=0.1. The large learning rate makes the model bounce around instead of smoothly descending to the minimum error.

Part 3. Analysis

The gradient descent approach consistently provided better performance than the heuristic method. The sigmoid activation function allows for continuous error gradients, enabling more refined weight updates. The ability to track log loss over time provides valuable insight into the learning process that is not available with the binary step function. When the learning rate was 0.1 it proved optimal across different epoch counts. Lower learning rates (0.01) require more epochs to achieve comparable performance, while higher rates, like 1.0, introduce instability. Increasing epochs from 100 to 200 with lr = 0.1 yielded the best configuration, achieving 94% accuracy and the lowest error.